

W3 PRACTICE

Express Basics + POST + Middleware



At the end of this practice, you can

- ✓ **Create** and run a express.js HTTP server ✓ **Implement** route handling using express.js ✓ Parse form data from POST requests with middleware.
- ✓ Apply middleware concept to logging



Get ready before this practice!

- ✓ **Read** the following documents to understand the nature of Express.js:
<https://expressjs.com/>
- ✓ **Read** the following documents to know more about Express.js's built-in middleware's:
<https://expressjs.com/en/resources/middleware.html>
- ✓ **Read** the following documents to understand MDN: HTTP POST:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST>
- ✓ **Read** the following documents to array filter: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter



How to submit this practice?

- ✓ Once finished, push your **code to GITHUB** ✓ Join the **URL of your GITHUB** repository on LMS



EXERCISE 1 – Refactoring

Goals

- ✓ Take advantage of Express.js framework's flexibility and minimalism ✓
- Refactor code from node.js's built-in HTTP Module



Refactor the source code of EXERCISE 2 & 3 in Week 2 to Express.js

Q1 – What challenges did you face when using the native http module that Express.js helped you solve?

Challenges that I face when using the native http module are that module required writing complex, nested if/else statements just to handle different URLs and HTTP methods. It also forced me to manually catch low-level data streams chunk-by-chunk to read form submissions, Express.js solved these challenges by providing clean routing functions (app.get and app.post), automatic body-parsing middleware, and simple response methods like res.send() that automatically manage content types and headers behind the scenes.

Q2 – How does Express simplify route handling compared to the native HTTP server?

Express.js simplify route handling with a clean, declarative routing system:

- Dedicated Methods: It isolates logic by HTTP verbs using explicit methods like app.get(), app.post().
- Pattern Matching: It maps requests directly to specific URL string paths, eliminating nested conditional chains.

Q3 – What does middleware mean in Express, and how would you replicate similar behavior using the native module?

Middleware functions are functions that run before your route handler. They can:

- Modify the req and res objects
- End the request-response cycle (res.send, etc.)
- Call the next middleware using next()

To replicate this behavior using the native http module, we must build execution pipeline. Since the native module doesn't have a built-in stack to handle a sequence of functions, we must manually pass the req and res objects through a chain of wrapper functions, or invoke a custom next() mechanic.

EXERCISE 2 – API for Course Records

For this exercise you will start with a **START CODE (EX-2)**

- Goals** ✓ Understand Route Parameters (:param)
✓ Work with Query Parameters (?key=value)
✓ Implement Conditional Logic for Filtering
✓ Build Real-World Web API Behavior
✓ Practice Defensive Programming

Context

You are building a backend API for a university's course catalog. Each course has the following fields

```
{
  "id": "CSE101",
  "title": "Introduction to Computer Science",
  "department": "CS",
  "level": "undergraduate",
  "credits": 3,
  "instructor": "Dr. KimAng",
  "semester": "fall"
}
```

Q1 - Create a route

```
GET /departments/:dept/courses
```

EXAMPLE

```
/departments/CSE/courses
```

Q2 - Accept query parameters to filter the result:

- `level` → e.g., undergraduate, graduate
- `minCredits` → integer
- `maxCredits` → integer
- `semester` → fall, spring, etc.
- `instructor` → partial match

EXAMPLE

```
/departments/CSE/courses?level=undergraduate&minCredits=2&semester=fall
```

Q3 - Return a JSON array of courses that match:

- The `:dept` from the route parameter
- The filter criteria from query parameters

Q4 – Handle Edge Cases

- **Invalid credit ranges** (minCredits > maxCredits)
- **No matching courses**
- **Missing or unsupported** query parameters (ignore them silently)

EXAMPLES

REQUEST
/departments/CSE/courses?level=undergraduate&minCredits=3&instructor=KimAng
RESPONSE
<pre>{ "results": [{ "id": "CSE101", "title": "Introduction to Data Science", "department": "CSE", "level": "undergraduate", "credits": 3, "instructor": "Dr. KimAng", "semester": "fall" }], "meta": { "total": 1 } }</pre>

Test cases

- http://localhost:3000/departments/CSE/courses
- http://localhost:3000/departments/CSE/courses?level=undergraduate
- http://localhost:3000/departments/CSE/courses?minCredits=4
- http://localhost:3000/departments/CSE/courses?instructor=smith&semester=fall

EXERCISE 3 – Enhance an API with Middleware

Goal

Your goal is to modularize and secure your course filtering API using **Express middleware**. Middleware helps keep your code clean, reusable, and extensible.

Q1 - Create a middleware function that logs the following for every request:

- HTTP method (GET, POST, etc.)
- Request path (e.g., /departments/CSE/courses)
- Query parameters

- Timestamp in ISO format

✓ **Apply this middleware globally** so it logs **all incoming requests** to the server.

Q2 - Create a route-specific middleware to **validate query parameters**:

- If minCredits or maxCredits are present, ensure they are valid integers.
- If minCredits > maxCredits, return 400 Bad Request with an error message. ✓

Apply this middleware only to the /departments/:dept/courses route.

Q3 – (*Bonus*) Token-Based Authentication Middleware

Simulate basic API security:

- Require a token query parameter (e.g., ?token=xyz123)
- If the token is missing or incorrect, respond with 401 Unauthorized.

✓ This middleware can be applied **either globally or to specific routes**.

Deliverables

- logger.js – contains your logging middleware.
- validateQuery.js – contains your validation middleware.
- auth.js (optional) – contains your token authentication middleware.
- server.js – where you apply middleware and define the course filtering route.

Test cases

```
GET /departments/CSE/courses?minCredits=abc
```

→ should return 400 Bad Request

```
GET /departments/CSE/courses?minCredits=4&maxCredits=2
```

→ should return 400 Bad Request

→ should succeed if token middleware is active

REFLECTIVE QUESTIONS

For this part, submit it in separate PDF files

Middleware & Architecture

1. What are the advantages of using middleware in an Express application?

The advantages of using middleware in an Express application:

- Separation of concerns: Logging, authentication, and validation each live in their own function. The route handler only has to think about business logic.
- Reusability: A single `validateQuery` function can guard every route that accepts credit filters.
- Composability: Middleware stacks like building blocks. Adding rate-limiting later means inserting one line without touching existing handlers.
- Testability: Each middleware function accepts `(req, res, next)` and returns a predictable result easy to unit-test in isolation with mock objects.
- Early termination: Returning a 401 or 400 in middleware means the route handler is never reached, keeping the happy-path code clean.

2. How does separating middleware into dedicated files improve the maintainability of your code?

Separating middleware into dedicated files improve the maintainability of code:

- Discoverability: A new developer can open the project and immediately understand the system by reading the file names alone.
- Independent iteration: Changing how tokens are validated does not risk breaking the logging logic because they are in different files.
- Reduced merge conflicts: Teams editing separate files concurrently avoid stepping on each other in version control.
- Easier testing: Each file exports a single function, which is the simplest unit to import into a test suite.

3. If you had to scale this API to support user roles (e.g., admin vs student), how would you modify the middleware structure?

The current auth middleware only checks whether a token is valid. Role support requires two additional concepts: token decoding and role-based authorisation.

- Decode the token in `auth.js`. Replace the hardcoded string comparison with a JWT verification step (e.g. `jwt.verify(token, SECRET)`). Attach the decoded payload — including a role field — to `req.user`.
- Create a new `authorise.js` middleware factory. A factory function like `authorise('admin')` returns a middleware that checks `req.user.role` against the required role.
- Apply per route. `app.delete('/courses/:id', auth, authorise('admin'), handler)` — students reach the GET routes; only admins reach destructive ones.
- Keep `auth.js` and `authorise.js` separate. Authentication (who are you?) and authorisation (what can you do?) are distinct concerns.

Query Handling & Filtering

4. How would you handle cases where multiple query parameters conflict or are ambiguous (e.g., `minCredits=4` and `maxCredits=3`)?

A conflict means the filter constraints produce an impossible set. The correct strategy is to detect it early and respond clearly, which is exactly what `validateQuery.js` does:

- Validate before use. The middleware catches the conflict before the route handler runs, so the filtering code never receives logically broken input.

5. What would be a good strategy to make the course filtering more user-friendly (e.g., handling typos in query parameters like “falll” or “dr. smtih”)?

Exact string matching is brittle. Two complementary strategies improve resilience:

- Fuzzy matching with a similarity score: Libraries like `fuse.js` (client) or `natural` (Node) compute edit distance. An instructor query of `'dr. smtih'` matches `'Dr. Smith'` if the score is above a threshold (e.g. 0.6).
- Normalisation before comparison: Lowercase both sides, strip extra whitespace, and remove punctuation before the filter runs. `'falll' → 'falll'` still fails, but `'Fall'`, `'FALL'`, and `'fall '` all match correctly.
- Enum validation with suggestions: For fields like semester or level, maintain an allow-list. If the value is not in the list, return a 400 that includes the closest valid value: `'Did you mean: fall?'`
- Partial / substring matching: Use `.includes()` (already in the codebase for instructor) so `'smith'` finds `'Dr. Smith'` without a full match.

Security & Validation

6. What are the limitations of using a query parameter for authentication (e.g., `?token=xyz123`)? What alternatives would be more secure?

Passing credentials in the query string is convenient for development but introduces serious security risks in production:

- Tokens appear in server logs. Every proxy, load balancer, and web server logs the full URL by default — including the token value.
- Tokens appear in browser history. Any user who visits a URL with a token can find it later in their history.
- Tokens appear in Referer headers. When a page with a token URL links to an external resource, the token leaks via the Referer HTTP header.
- No expiry or rotation. A hardcoded string never expires. If it leaks, it is valid forever until manually changed.

7. Why is it important to validate and sanitize query inputs before using them in your backend logic?

User-supplied input is untrusted by definition. Using it without validation or sanitisation opens the application to several categories of attack and logic error:

- Type coercion bugs. `Number('abc')` returns `NaN`. Comparing `NaN >= 3` is always false, silently returning all courses regardless of the filter.

Abstraction & Reusability

8. Can any of the middleware you wrote be reused in other projects? If so, how would you package and document it?

All three middleware functions are good candidates for extraction into a shared package because they have no domain-specific logic:

- `logger.js` logs method, path, query, and timestamp for any Express app. Zero coupling to course data.
- `auth.js` token verification is universally applicable; only the token value and its source (env var) need to be parameterised.
- `validateQuery.js` the integer-validation and min/max logic could be generalised into a `validateIntegerRange(minParam, maxParam)` factory.

Packaging steps:

- Convert to factory functions so callers can configure behaviour: `createLogger({ format: 'json' })`, `createAuth({ token: process.env.API_KEY })`.

- Publish as an npm package with a clear `package.json` (name, version, main, exports) and a README describing each export with usage examples.

- Write JSDoc comments on every exported function so editors show inline documentation.

- Add unit tests (Jest or Vitest) that mock `req`, `res`, and `next`, so consumers can verify the package works before installing.

9. How could you design your route and middleware system to support future filters (e.g., course format, time slot)?

The key is to make adding a new filter a data change, not a code change. This is achieved through a filter schema that drives both validation and the query logic:

- Define a filter schema object. Each key maps to its type, validator, and comparator. New filters (format, timeSlot) are added by extending the schema the middleware and handler loop over it automatically.

- Centralise validation in `validateQuery`. Instead of a chain of if-statements, iterate the schema and validate each present param. Any param not in the schema returns a 400 'unknown filter' response.

- Generate filter functions dynamically. The route handler iterates active filters, applies each comparator from the schema, and chains them with `.filter()`.

- Keep `validateQuery` as the single gatekeeper. The route handler trusts that any value that reaches it has already been validated no defensive coding inside the handler.

Bonus – Real-World Thinking

10. How would this API behave under high traffic? What improvements would you need to make for production readiness (e.g., rate limiting, caching)?

The current API loads all courses into memory and filters them on every request. Under low traffic this is fine. Under high traffic, several bottlenecks and risks emerge:

- No rate limiting. A single client can flood the server with thousands of requests per second, starving other users. Fix: add `express-rate-limit` middleware with a per-IP window.

- No caching. Identical queries recompute the same results. Fix: cache responses in Redis with a short TTL (e.g. 30 seconds). Cache-Control headers let CDNs and clients cache GET responses too.

- In-memory data. The courses array is hardcoded. At scale it would live in a database (PostgreSQL, MongoDB). Database queries should be parameterised and indexed on department and credits.

- No request compression. Large JSON responses waste bandwidth. Fix: add the compression middleware to gzip responses.
- No error monitoring. Unhandled exceptions crash the process silently. Fix: add a global error-handling middleware (`app.use((err, req, res, next) => ...)`) and integrate a monitoring service (Sentry, Datadog).
- Single process. Node.js is single-threaded. Fix: use the cluster module or deploy behind a process manager (PM2) to fork one worker per CPU core.
- No HTTPS enforcement. In production all traffic must be encrypted. Fix: terminate TLS at a load balancer or reverse proxy (nginx, AWS ALB) and redirect HTTP to HTTPS.
- No helmet.js. Express does not set secure HTTP headers by default. Fix: `app.use(helmet())` adds Content-Security-Policy, X-Frame-Options, and others in one line.